

综合实验报告

实 验 名 称: AllRecipes 食谱网站数据库方案设计与实现

实验项目类型 综合实验—学期项目

实 验 室 云计算研究中心实验室 520

所属课程名称 《数据库》

实 验 日 期 2025.9 ----2025.12

小组成员（签名） 倪伟丰 王宇阳 史家和

课程序号 0827

成 绩

【实验目的】

综合运用课程所学的知识点进行具体背景的数据库方案设计和实现。

【实验要求】

至少完成如下设计和实现内容：

1、设计部分

项目背景介绍

ER 图设计（概念模型）

数据库库表设计（逻辑模型）

规范化

常见操作归纳

完整性约束方案设计

表的详细设计（物理模型）

视图设计方案

数据库安全方案的设计

2、实现部分

表格创建（可同时包括约束条件的创建）

数据录入

视图创建和使用

常见操作的实例演示

数据库安全的实现

3、总结

组员分工

难点与不足分析

收获或体会及建议

【实验环境】

Oracle 11g 或更高版本

【实验内容】

根据以上要求，后附报告，展示综合实验-学期项目具体设计和实现的内容。

【实验报告的提交要求】

用<ALT>+<Print Screen>快捷键，将相应的证据结果屏幕复制下来，记录在本综合实验报告中；

要求排版及文字大小一致、清楚、整洁、美观；

要求正文楷体五号，单倍行距；

请将本报告整体双面打印，不能只打印附件。

【实验报告提交方式】

将本实验报告存放在“综合-XXXXXXXXX.docx”文件中，其中“XXXXXXXXX”是组员名字，并在 2025 年 12 月 28 日前通过 canvas 系统提交该电子版文件（组长提交即可），同时打印版放在科研实验大楼 835 办公室办公桌上或门口兜袋（每组提交一份）。

《AllRecipes 食谱网站数据库方案设计与实现》报告

目录

1 设计部分	3
1.1 项目背景介绍	3
1.1.1 AllRecipes 网站概述	3
1.1.2 核心业务数据	3
1.1.3 业务流程分析	4
1.2 ER 图设计（概念模型）	5
1.2.1 ER 图	5
1.2.2 几个关键问题的讨论与思考	8
1.3 数据库库表设计（逻辑模型）	8
1.4 规范化	8
1.5 常见操作归纳	8
1.5.1 查询操作（SELECT）	9
1.5.2 数据变更操作	9
1.6 完整性约束方案设计	10
1.7 表的详细设计（物理模型）	10
1.7.1 表的分类和数量统计	10
1.7.2 核心表详细设计	10
1.8 视图设计方案	18
1.9 数据库安全方案的设计	19
1.9.1 数据库人员	19
1.9.2 权限分配	20
2 实现部分	20
2.1 表格创建（可同时包括约束条件的创建）	20
2.1.1 核心表 USERS 与 RECIPES 的创建	20
2.1.2 转表 RECIPE_INGREDIENTS 创建	22
2.1.3 有自引用的表	23
2.2 数据录入	24
2.3 视图创建和使用	25
2.3.1 视图创建概览	25
2.3.2 用户相关视图	25
2.3.3 食谱相关视图	27

2.3.4	社交互动视图	28
2.3.5	分析报表视图	29
2.4	常见操作的实例演示	29
2.4.1	插入操作	29
2.4.2	查询操作	31
2.4.3	更新操作	33
2.4.4	删除操作	35
2.5	数据库安全的实现	35
2.5.1	数据库用户角色划分	35
2.5.2	精细化权限分配	36
3	总结	37
3.1	组员分工	37
3.2	难点与不足分析	37
3.2.1	难点	37
3.2.2	不足	38
3.3	收获或体会及建议	38

1 设计部分

1.1 项目背景介绍

1.1.1 AllRecipes 网站概述

AllRecipes 是全球最大的食谱分享平台之一，拥有数百万用户和数百万条食谱。该网站界面简洁直观，拥有丰富的食谱内容和强大的社区功能。以下为五点核心功能：

- **食谱发布与分享**：用户可以上传自己创作的食谱，包括食材清单、烹饪步骤、烹饪时间、难度等级等信息；同时支持创作者上传对应的菜品图片。
- **多维度食谱搜索与浏览**：用户可以通过关键词、菜系、食材、难度级别等多个维度进行灵活搜索，找到适合自己的食谱。
- **用户交互与社交功能**：包括食谱评价、评论、收藏、关注其他用户等丰富的互动方式。
- **个性化食谱管理**：用户可以创建多个食谱收藏清单、购物清单、膳食计划等，使得饮食更有规划。
- **健康饮食支持**：支持用户录入过敏原信息、查看食谱营养成分（热量 / 蛋白质 / 膳食纤维等），并推荐适配饮食需求的安全食谱，帮助用户实现过敏规避、减脂控糖、营养均衡等饮食计划。

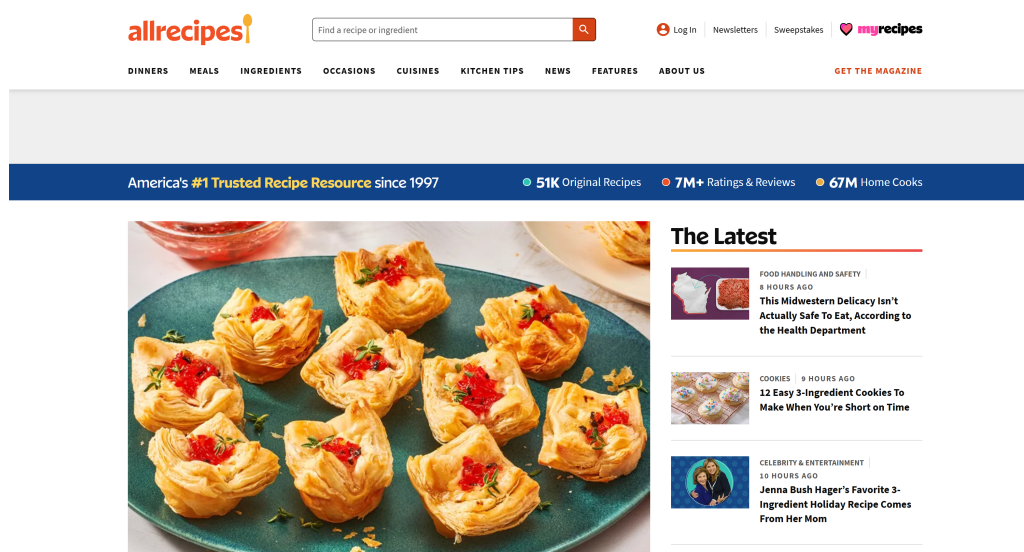


图 1: AllRecipes 网站概览

1.1.2 核心业务数据

AllRecipes 的主要数据对象包括：

表 1: 核心业务数据对象

数据对象	描述	关键属性
用户 (Users)	平台使用者	用户名、邮箱、密码、个人资料、粉丝数、账户状态
食谱 (Recipes)	烹饪菜谱	食谱名称、描述、菜系、难度、烹饪时间、评分、浏览数
食材 (Ingredients)	烹饪所需原料	食材名称、分类、描述、过敏原信息
膳食计划 (MealPlans)	用户的一周/一月食谱安排	计划名称、时间范围、公开状态
购物清单 (ShoppingLists)	用户需要购买的食材	清单名称、食材列表、购买状态
评价/评论 (Rating/Comment)	用户对食谱的反馈	评分、评论文本、有用性评分
社交关系 (Followers)	用户之间相互关注	关注时间、粉丝数、活动流

1.1.3 业务流程分析

1.1.3.1 食谱发布流程

用户注册 → 登录 → 创建食谱 → 添加基本信息 → 添加食材 → 设置烹饪步骤 → 上传图片 → 验证和预览 → 发布

1.1.3.2 食谱浏览与评价流程

用户浏览 → 搜索（按菜系/食材/难度等）→ 查看食谱详情 → 查看评价和评论 → 添加到收藏清单 → 评价或评论

1.1.3.3 膳食规划流程

查看推荐食谱 → 创建膳食计划 → 为每天安排食谱 → 系统整合购物清单 → 生成购物清单 → 管理购物进度

1.1.3.4 社区互动流程

浏览用户资料 → 关注用户 → 查看关注用户的食谱 → 与用户互动（评价、评论、私信等）

1.2 ER 图设计 (概念模型)

1.2.1 ER 图

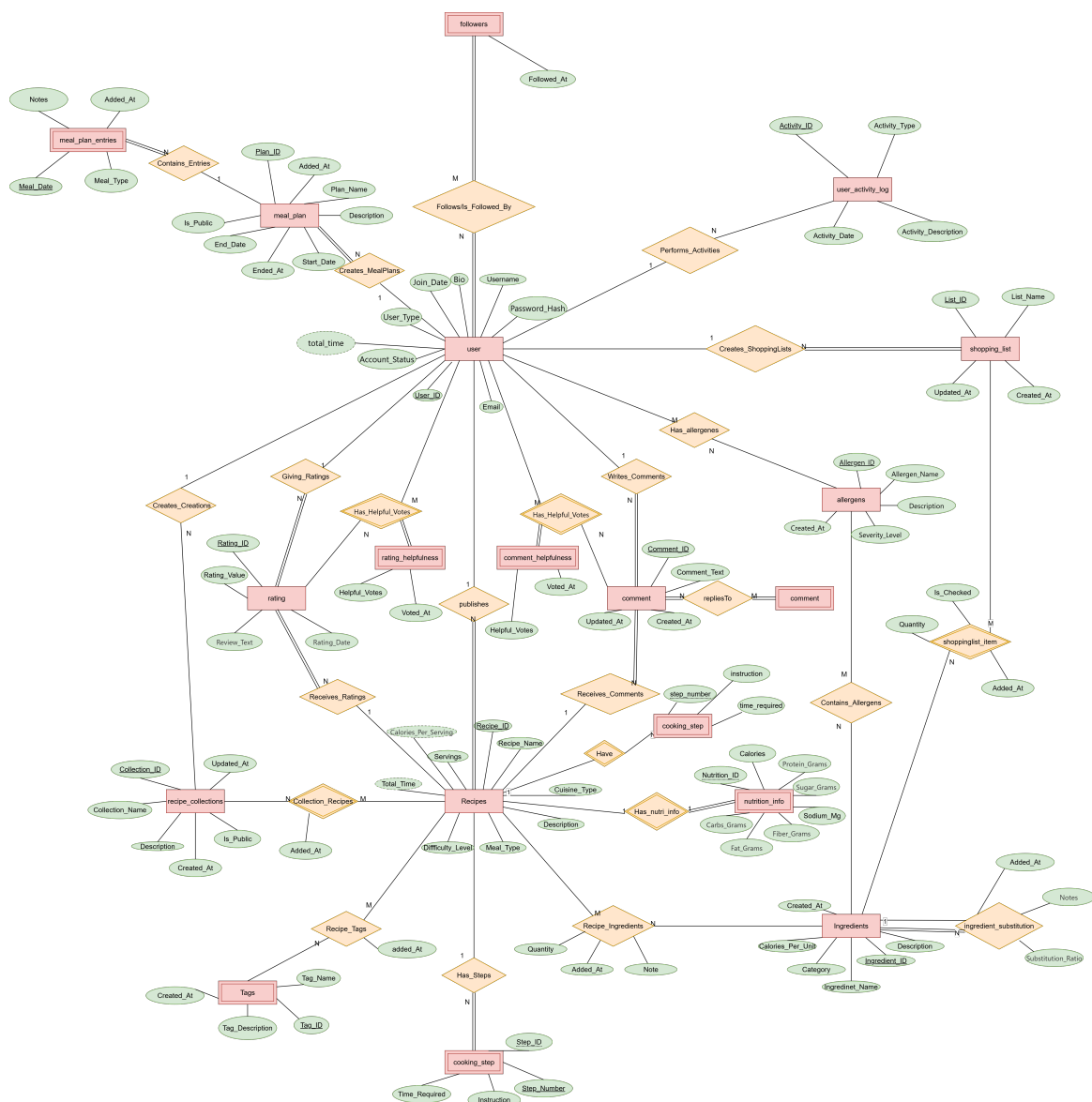


图 2: ER 图设计 (概念模型)

由于完整 ER 图较为庞大, 为清晰展示细节, 将其拆分为四个部分进行展示:

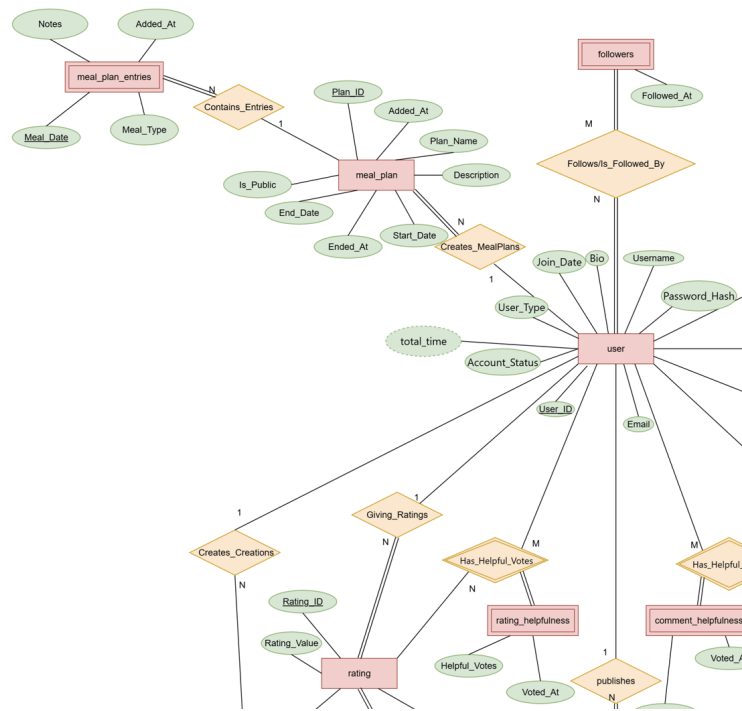


图 3: ER 图左上

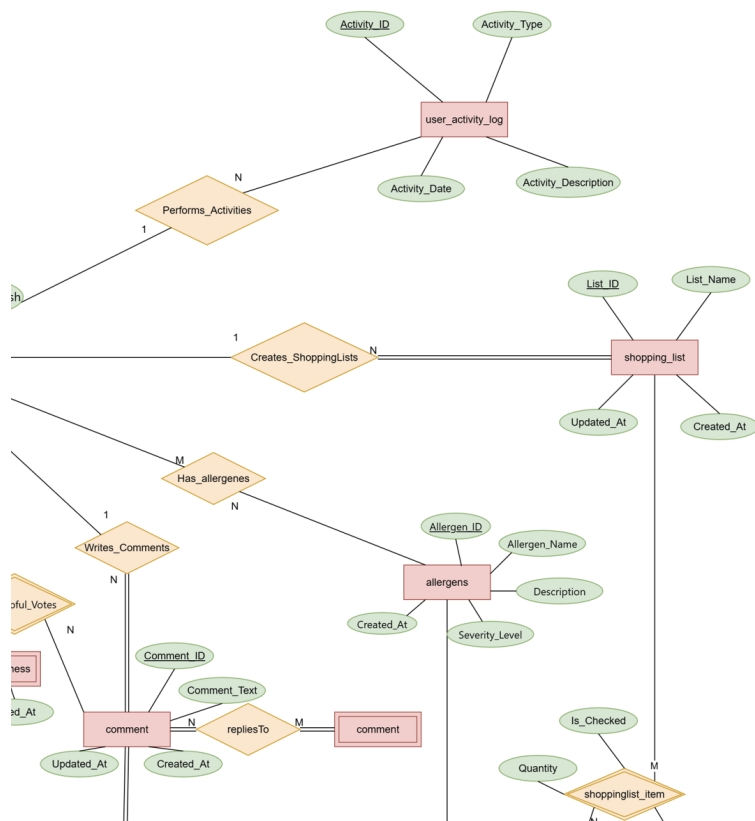


图 4: ER 图右上

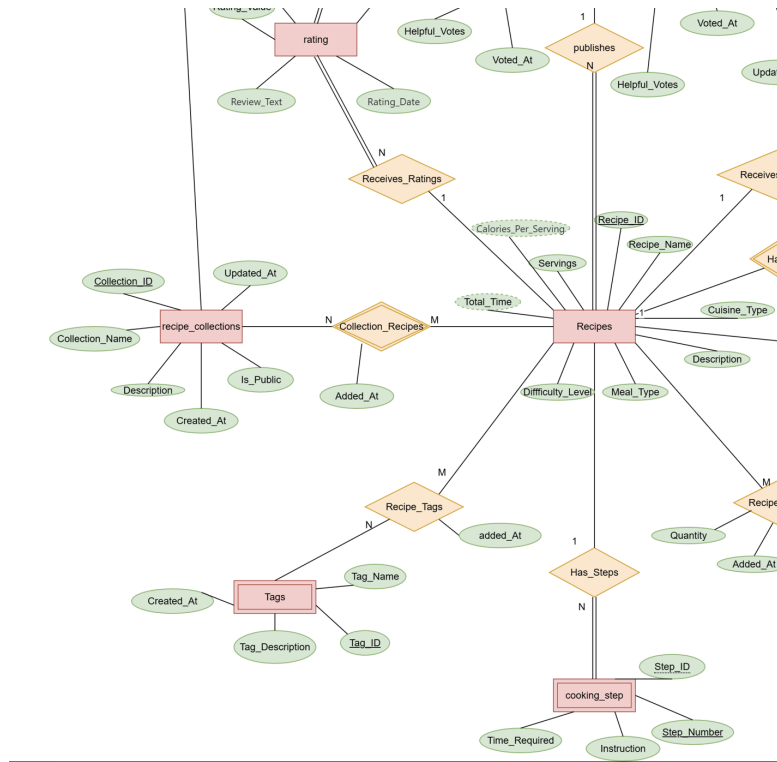


图 5: ER 图左下

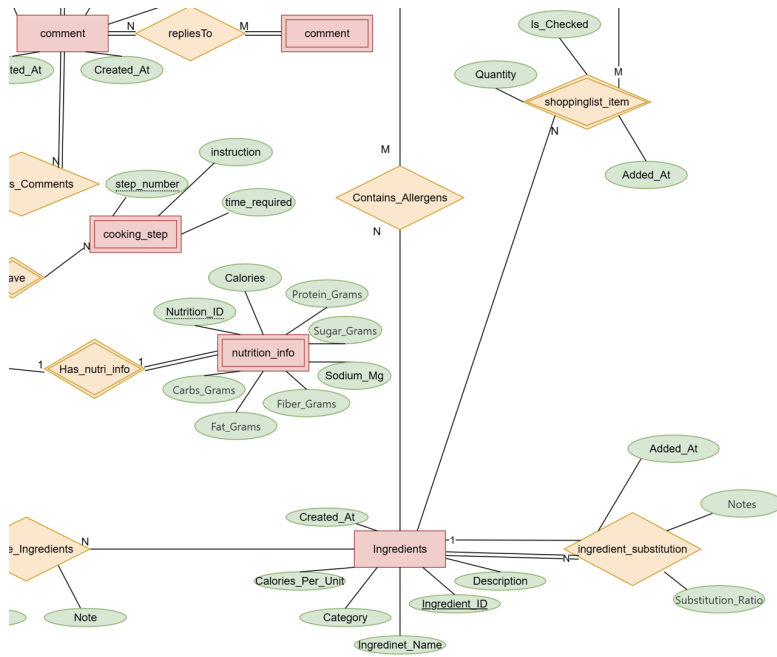


图 6: ER 图右下

1.2.2 几个关键问题的讨论与思考

关于购物车模型的处理 上课在 ER 图小组展示的时候有特别提到购物车模型的处理方法。这个模型中有实体用户以及商品参与，用户与商品是多对多关系，两边都是部分参与。针对这种情况，直接把“购物车”当作一种关系。

我们发现，在 AllRecipe 上广泛存在这个购物车模型。比如，用户的膳食计划，用户的购买清单，用户的食谱收藏夹。但是，我们发现在 AllRecipe 上膳食计划、购物清单、食谱收藏夹也可以有自己的属性（比如收藏夹名称，收藏夹简介之类的），这样一来，使用上课的这种模型会导致转表之后的关系表不符合 2NF。我们需要另外一个属性，比如购物清单的编号，来表示一个原料到底被添加到了哪一个用户的哪一个购物清单中。这时主键是用户 ID、原料 ID 和清单 ID 形成的联合主键。但是，比如清单的名称、描述等属性只函数依赖于清单 ID，造成了部分函数依赖。

所以，我们设计了 RECIPE_COLLECTIONS, COLLECTION_RECIPES, SHOPPING_LISTS, SHOPPING_LIST_ITEMS, MEAL_PLANS, MEAL_PLAN_ENTRIES 这 6 个表。我们把食谱收藏夹、购买清单、膳食计划都当作一个实体。以食谱收藏夹为例，食谱收藏夹与食谱是多对多关系，用户与食谱收藏夹是 1 对多关系。这样子食谱收藏夹可以有自己的名称和简介，食谱收藏夹中将用户 ID 作为外键，收藏夹由多对多关系与食谱链接。

评价模型的处理 同样的我们这里还有一个比较复杂但是比较合理的设计，就是 Ratings 和 Comments。它们都是用户对于食谱的评价，区别在于，一个用户只能对一个食谱 Rate 一次，但是可以 Comment 多次，而且 Comment 可以和 Comment 有自引用关系。

和购物车不同，用户和 Comment 是 1 对多关系，Comment 和 Recipe 也是一对多关系。所以 User_id 和 Recipe_id 统统拿来作外键了。按理来讲，我们只需要将 Comment 当作一个关系嫁接用户和 Recipe 就好了，但是，我们把它当作一个实体。不仅是因为 Comment 存在自引用的难题，而且 AllRecipe 的网站给我们出了个难题，我们发现用户居然可以给别人的评论打分！所以，Comment 和 User 之间还有一个关系，而且这个关系是多对多的，不得不多转一个表，相当痛苦。

以上两个部分是我们 ER 图设计交流课后又思考讨论了很久的部分。我们尽力让设计出的 ER 图符合实际业务的语义，同时不要给我们转表留下太多隐患。哎。

1.3 数据库库表设计（逻辑模型）

由于篇幅所限，具体的参考物理部分实现部分。

1.4 规范化

本设计严格遵循 BCNF (Boyce-Codd Normal Form) 规范：

- **第一范式 (1NF)**：所有字段包含原子值，不可再分拆；
- **第二范式 (2NF)**：消除了部分函数依赖；
- **第三范式 (3NF)**：消除了所有传递依赖；
- **BCNF**：满足主键是唯一的候选键。

1.5 常见操作归纳

这是我们归纳的常见操作，但是后续的具体操作演示，我们只实现表格中的部分操作

1.5.1 查询操作 (SELECT)

表 2: 查询操作归纳

类别	操作名称	描述/目的
搜索功能	按菜系搜索食谱	根据菜系/发布状态筛选食谱, 按评分/浏览量排序
	按原材料搜索食谱	获取指定原材料以及其替代品的所有未删除食谱
用户功能	查询用户的所有食谱	获取指定用户发布的所有未删除食谱
	查询用户收藏的食谱	获取指定用户收藏的食谱列
食谱功能	查询食谱详情	获取食谱的基本信息: 作者、营养成分及标签等
	查询食谱食材	列出食谱所需的所有食材、用量及单位
	查询烹饪步骤	按顺序获取食谱的制作步骤和图片
评价功能	查询食谱评价	获取食谱的评分、评论内容及有用性
	查询食谱评论	递归查询食谱的评论树 (含回复)
社交功能	查询关注/粉丝	获取用户的关注列表和粉丝列表
	关注者食谱推荐	推荐用户关注的创作者发布的食谱
	特定食材搜索	查找包含指定食材 (如鸡蛋、面粉) 的食谱

1.5.2 数据变更操作

表 3: 数据变更操作归纳

操作类型	类别	操作名称	描述/目的
插入 (INSERT)	用户与食谱	新用户注册	创建新的用户账户信息
		发布新食谱	插入食谱、食材、步骤、营养信息
	交互	用户评价/评论	添加评分、评论内容, 并更新统计数据
		收藏食谱	将食谱添加到用户收藏
		关注用户	建立用户间的关注关系
	计划与清单	创建膳食计划	建立新的膳食计划记录
		添加计划表	向膳食计划中添加食谱
		创建购物清单	生成清单并导入计划所需的食谱
	设置	添加过敏	记录用户的过敏食材信息
更新 (UPDATE)	编辑	更新食谱信息	修改食谱详情及更新时间
		逻辑删除	将食谱/评论标记为删除状态
	状态	禁用账户	修改用户账户状态为挂起
		购物清单勾选	标记清单中的物品为已购买
删除 (DELETE)	清理	删除评价/收藏	物理删除评价记录或取消收藏
		清空购物清单	删除指定清单内的所有条目
		取消关注	移除关注关系
		撤销投票	取消对评价/评论“有用”标记
		移除过敏源	删除用户的过敏记录

1.6 完整性约束方案设计

具体内容由于篇幅所限，参考物理实现部分

1.7 表的详细设计（物理模型）

1.7.1 表的分类和数量统计

模块	表名	数量
用户功能	USERS, INGREDIENTS, UNITS, ALLERGENS, TAGS, USER_ ALLERGIES	6
食谱功能	RECIPES, RECIPE_INGREDIENTS, COOKING_STEPS, NUTRITION_INFO, INGREDIENT_ALLERGENS, RECIPE_TAGS, INGREDIENT_SUBSTITUTIONS	7
评价与社交功能	RATINGS, RATING_HELPFULNESS, COMMENTS, COMMENT_ HELPFULNESS, FOLLOWERS, USER_ACTIVITY_LOG	6
个性化功能	RECIPE_COLLECTIONS, COLLECTION_RECIPES, SHOPPING_ LISTS, SHOPPING_LIST_ITEMS, MEAL_PLANS, MEAL_PLAN_ ENTRIES	6
总计		25

表 4: 表的分类和数量统计

1.7.2 核心表详细设计

关于部分派生属性的处理 我们考虑到有部分实体具有派生属性，然而，随着数据的不断加入，有些派生属性的计算有比较大的系统开销，因此，有些派生属性我们也将其当作一个基本属性加入到表中。这些属性将在固定的时间由系统一次性自动更新。

还有些关键派生属性的计算需要设计多个表，一般随着它们的更新而更新，其被更新的次数也比较少。为了简化查询代码，我们也把它们当作基本属性设计到物理表里。我们的设想是可以使用触发器，在这些表每次更新之后，也自动更新这些派生属性。比如 RECIPES 中的烹饪时长是派生属性，但是，这个属性可以使用 RECIPES 和 RECIPE_STEPS 进行计算。我们希望能设计一个触发器，每当 RECIPE_STEPS 中某一步骤的时长被更新了，相应地在 RECIPES 中也自动更新烹饪时长。

用户功能

字段名	数据类型	非空	约束	说明
USER_ID	NUMBER(10)	√	PK	用户主键，自增
USERNAME	VARCHAR2(50)	√	UK	用户名，唯一
EMAIL	VARCHAR2(100)	√	UK	邮箱，唯一，用于登录和验证
PASSWORD_ HASH	VARCHAR2(255)	√		加密后的密码哈希值 (暂时未实现)
FIRST_NAME	VARCHAR2(50)			用户名称
LAST_NAME	VARCHAR2(50)			用户姓氏

BIO	VARCHAR2(500)			个人简介或专业资格描述
PROFILE_IMAGE	VARCHAR2(255)			头像图片 URL
JOIN_DATE	DATE	√		注册日期
LAST_LOGIN	TIMESTAMP			最后登录时间
ACCOUNT_STATUS	VARCHAR2(20)	√	CK	active/inactive/suspended
TOTAL_FOLLOWERS	NUMBER(10)	√	DF=0	粉丝数量（派生属性）
TOTAL_RECIPES	NUMBER(10)	√	DF=0	发布的食谱数量（派生属性）
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建记录时间
UPDATED_AT	TIMESTAMP	√	DF=SYSDATE	最后修改时间

2. INGREDIENTS 表（食材表）

字段名	数据类型	非空	约束	说明
INGREDIENT_ID	NUMBER(10)	√	PK	食材主键
INGREDIENT_NAME	VARCHAR2(100)	√	UK	食材名称（如：番茄、鸡蛋）
CATEGORY	VARCHAR2(50)	√		食材分类（蔬菜、肉类、调味料等）
DESCRIPTION	VARCHAR2(255)			食材描述与特点
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建时间

3. UNITS 表（单位表）

字段名	数据类型	非空	约束	说明
UNIT_ID	NUMBER(10)	√	PK	单位主键
UNIT_NAME	VARCHAR2(50)	√	UK	单位名称（如：克、毫升、杯）
ABBREVIATION	VARCHAR2(20)			单位缩写（如：g, ml, cup）
DESCRIPTION	VARCHAR2(100)			单位描述和转换信息
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建时间

4. ALLERGENS 表（过敏原表）

字段名	数据类型	非空	约束	说明
ALLERGEN_ID	NUMBER(10)	√	PK	过敏原主键

ALLERGEN_NAME	VARCHAR2(100)	√	UK	过敏原名称（花生、坚果、乳制品等）
DESCRIPTION	VARCHAR2(255)			过敏原详细描述和影响
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建时间

5. TAGS 表（标签表）

字段名	数据类型	非空	约束	说明
TAG_ID	NUMBER(10)	√	PK	标签主键
TAG_NAME	VARCHAR2(50)	√	UK	标签名称（如：素食、低脂、快手菜）
TAG_DESCRIPTION	VARCHAR2(255)			标签描述
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建时间

食谱核心表

6. RECIPES 表（食谱表）

字段名	数据类型	非空	约束	说明
RECIPE_ID	NUMBER(10)	√	PK	食谱主键
USER_ID	NUMBER(10)	√	FK	创建者用户 ID
RECIPE_NAME	VARCHAR2(200)	√		食谱名称
DESCRIPTION	VARCHAR2(1000)			详细描述
CUISINE_TYPE	VARCHAR2(50)			菜系（中式、西式、日式等）
MEAL_TYPE	VARCHAR2(20)		CK	breakfast/lunch/dinner/snack/dessert
DIFFICULTY_LEVEL	VARCHAR2(20)		CK	easy/medium/hard
TOTAL_TIME	NUMBER(5)	√	CK>0	总时间（分钟）
SERVINGS	NUMBER(5)			份数
CALORIES_PER_SERVING	NUMBER(10)			每份热量
IMAGE_URL	VARCHAR2(255)			食谱主图 URL
IS_PUBLISHED	VARCHAR2(1)	√	CK	Y/N，是否发布
IS_DELETED	VARCHAR2(1)	√	CK	Y/N，逻辑删除
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建时间
UPDATED_AT	TIMESTAMP	√	DF=SYSDATE	最后更新时间

7. RECIPE_INGREDIENTS 表（食谱食材关联表 - 多对多）

字段名	数据类型	非空	约束	说明
-----	------	----	----	----

RECIPE_ID	NUMBER(10)	√	PK1, FK	食谱 ID, 联合主键第一部分
INGREDIENT_ID	NUMBER(10)	√	PK2, FK	食材 ID, 联合主键第二部分
UNIT_ID	NUMBER(10)	√	FK	计量单位 ID
QUANTITY	NUMBER(10,2)	√		食材用量
NOTES	VARCHAR2(255)			特殊说明 (如: 切碎、预先煮沸)
ADDED_AT	TIMESTAMP	√	DF=SYSDATE	添加时间

8. COOKING_STEPS 表 (烹饪步骤表)

字段名	数据类型	非空	约束	说明
STEP_ID	NUMBER(10)	√	PK	步骤局部键
RECIPE_ID	NUMBER(10)	√	PK, FK	食谱 ID
STEP_NUMBER	NUMBER(5)	√	UK (RECIPE_ID)	步骤序号 (1,2,3...)
INSTRUCTION	VARCHAR2(1000)	√		详细操作说明
TIME_REQUIRED	NUMBER(5)			该步骤所需时间 (分钟)
IMAGE_URL	VARCHAR2(255)			步骤配图 URL

9. NUTRITION_INFO 表 (营养信息表)

字段名	数据类型	非空	约束	说明
NUTRITION_ID	NUMBER(10)	√	PK	营养信息局部键
RECIPE_ID	NUMBER(10)	√	PK, FK, UK	食谱主键
CALORIES	NUMBER(10)			热量 (卡路里)
PROTEIN_GRAMS	NUMBER(10,2)			蛋白质 (克)
CARBS_GRAMS	NUMBER(10,2)			碳水化合物 (克)
FAT_GRAMS	NUMBER(10,2)			脂肪 (克)
FIBER_GRAMS	NUMBER(10,2)			纤维 (克)
SUGAR_GRAMS	NUMBER(10,2)			糖 (克)
SODIUM_MG	NUMBER(10)			钠 (毫克)

10. INGREDIENT_ALLERGENS 表 (食材过敏原关联表 - 多对多)

字段名	数据类型	非空	约束	说明
INGREDIENT_ID	NUMBER(10)	√	PK1, FK	食材 ID, 联合主键第一部分
ALLERGEN_ID	NUMBER(10)	√	PK2, FK	过敏原 ID, 联合主键第二部分

11. INGREDIENT_SUBSTITUTIONS 表 (食材替代品关联表 - 多对多)

字段名	数据类型	非空	约束	说明
ORIGINAL_INGREDIENT_ID	NUMBER(10)	√	PK1, FK	原始食材 ID, 联合主键第一部分
SUBSTITUTE_INGREDIENT_ID	NUMBER(10)	√	PK2, FK	替代食材 ID, 联合主键第二部分
SUBSTITUTION_RATIO	NUMBER(5,2)			替代比例 (如: .5 表示 1:1.5)
NOTES	VARCHAR2(255)			替代说明
ADDED_AT	TIMESTAMP	√	DF=SYSDATE	添加时间

12. RATINGS 表 (食谱评价表)

字段名	数据类型	非空	约束	说明
RATING_ID	NUMBER(10)	√	PK	评价主键
USER_ID	NUMBER(10)	√	FK	评价者用户 ID
RECIPE_ID	NUMBER(10)	√	FK	被评价的食谱 ID
RATING_VALUE	NUMBER(3,2)	√	CK	评分 (0-5)
REVIEW_TEXT	VARCHAR2(1000)			评价文本
RATING_DATE	TIMESTAMP	√	DF=SYSDATE	评价时间

13. RATING_HELPFULNESS 表 (评价有用性投票表 - 多对多)

字段名	数据类型	非空	约束	说明
RATING_ID	NUMBER(10)	√	PK1, FK	被投票的评价 ID, 联合主键第一部分
USER_ID	NUMBER(10)	√	PK2, FK	投票者用户 ID, 联合主键第二部分
HELPFUL_VOTES	NUMBER(10)		DF=0	有用投票数
VOTED_AT	TIMESTAMP	√	DF=SYSDATE	投票时间

14. COMMENTS 表 (评论表)

字段名	数据类型	非空	约束	说明
COMMENT_ID	NUMBER(10)	√	PK	评论主键
RECIPE_ID	NUMBER(10)	√	FK	食谱 ID (外键)
USER_ID	NUMBER(10)	√	FK	评论者用户 ID
PARENT_COMMENT_ID	NUMBER(10)		FK	父评论 ID (自引用)
COMMENT_TEXT	VARCHAR2(1000)			评论内容
IS_DELETED	VARCHAR2(1)		DF='N'	逻辑删除标记
CREATED_AT	TIMESTAMP		DF=SYSTIMESTAMP	创建时间
UPDATED_AT	TIMESTAMP		DF=SYSTIMESTAMP	更新时间

15. COMMENT_HELPFULNESS 表 (评论有用性投票表)

字段名	数据类型	非空	约束	说明
COMMENT_ID	NUMBER(10)	√	PK, FK	被投票的评论 ID
USER_ID	NUMBER(10)	√	PK, FK	投票者用户 ID
VOTED_AT	TIMESTAMP		DF=SYSTIMESTAMP	投票时间

17. FOLLOWERS 表 (用户关注关系表- 多对多自引用)

字段名	数据类型	非空	约束	说明
USER_ID	NUMBER(10)	√	PK1, FK	被关注者用户 ID, 联合主键第一部分
FOLLOWER_USER_ID	NUMBER(10)	√	PK2, FK	关注者用户 ID, 联合主键第二部分
FOLLOWED_AT	TIMESTAMP	√	DF=SYSDATE	关注时间

18. USER_ALLERGIES 表 (用户过敏原关联表 - 多对多)

字段名	数据类型	非空	约束	说明
USER_ID	NUMBER(10)	√	PK1, FK	用户 ID, 联合主键第一部分
ALLERGEN_ID	NUMBER(10)	√	PK2, FK	过敏原 ID, 联合主键第二部分
ADDED_AT	TIMESTAMP	√	DF=SYSDATE	添加时间

19. RECIPE_TAGS 表（食谱标签关联表- 多对多）

字段名	数据类型	非空	约束	说明
RECIPE_ID	NUMBER(10)	√	PK1, FK	食谱 ID, 联合主键第一部分
TAG_ID	NUMBER(10)	√	PK2, FK	标签 ID, 联合主键第二部分
ADDED_AT	TIMESTAMP	√	DF=SYSDATE	添加时间

20. USER_ACTIVITY_LOG 表（用户活动日志表）

字段名	数据类型	非空	约束	说明
ACTIVITY_ID	NUMBER(10)	√	PK	活动记录主键
USER_ID	NUMBER(10)	√	FK	用户 ID
ACTIVITY_TYPE	VARCHAR2(50)			活动类型 (view/comment/rate 等)
RECIPE_ID	NUMBER(10)		FK	相关食谱 ID (可为空)
ACTIVITY_DESCRIPTION	VARCHAR2(255)			活动描述
ACTIVITY_DATE	TIMESTAMP	√	DF=SYSDATE	活动时间

个人管理表

21. RECIPE_COLLECTIONS 表（食谱收藏清单表）

字段名	数据类型	非空	约束	说明
COLLECTION_ID	NUMBER(10)	√	PK	清单主键
USER_ID	NUMBER(10)	√	FK	创建者用户 ID
COLLECTION_NAME	VARCHAR2(100)	√		清单名称
DESCRIPTION	VARCHAR2(500)			清单描述
IS_PUBLIC	VARCHAR2(1)	√	DF='Y',CK	是否公开 (Y/N)
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建时间
UPDATED_AT	TIMESTAMP	√	DF=SYSDATE	更新时间

22. COLLECTION_RECIPES 表（清单食谱关联 √- 多对多）

字段名	数据类型	非空	约束	说明
COLLECTION_ID	NUMBER(10)	√	PK1, FK	清单 ID, 联合主键第一部分

RECIPE_ID	NUMBER(10)	√	PK2, FK	食谱 ID, 联合主键第二部分
ADDED_AT	TIMESTAMP	√	DF=SYSDATE	添加时间

23. SHOPPING_LISTS 表 (购物清单表)

字段名	数据类型	非空	约束	说明
LIST_ID	NUMBER(10)	√	PK	购物清单主键
USER_ID	NUMBER(10)	√	FK	用户 ID
LIST_NAME	VARCHAR2(100)	√		清单名称
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建时间
UPDATED_AT	TIMESTAMP	√	DF=SYSDATE	更新时间

24. SHOPPING_LIST_ITEMS 表 (购物清单项目表- 多对多)

字段名	数据类型	非空	约束	说明
LIST_ID	NUMBER(10)	√	PK1, FK	购物清单 ID, 联合主键第一部分
INGREDIENT_ID	NUMBER(10)	√	PK2, FK	食材 ID, 联合主键第二部分
QUANTITY	NUMBER(10,2)			数量
UNIT_ID	NUMBER(10)		FK	单位 ID
IS_CHECKED	VARCHAR2(1)	√	DF='N',CK	是否已购 (Y/N)
ADDED_AT	TIMESTAMP	√	DF=SYSDATE	添加时间

25. MEAL_PLANS 表 (膳食计划表)

字段名	数据类型	非空	约束	说明
PLAN_ID	NUMBER(10)	√	PK	膳食计划主键
USER_ID	NUMBER(10)	√	FK	用户 ID
PLAN_NAME	VARCHAR2(100)	√		清单名称
DESCRIPTION	VARCHAR2(500)			计划描述
START_DATE	DATE	√	CK	开始日期
END_DATE	DATE	√	CK	结束日期
IS_PUBLIC	VARCHAR2(1)	√	DF='Y',CK	是否公开 (Y/N)
CREATED_AT	TIMESTAMP	√	DF=SYSDATE	创建时间
UPDATED_AT	TIMESTAMP	√	DF=SYSDATE	更新时间

26. MEAL_PLAN_ENTRIES 表 (膳食计划条目表- 多对多)

字段名	数据类型	非空	约束	说明
-----	------	----	----	----

PLAN_ID	NUMBER(10)	√	PK1, FK	膳食计划 ID, 联合主键第一部分
RECIPE_ID	NUMBER(10)	√	PK2, FK	食谱 ID, 联合主键第二部分
MEAL_DATE	DATE	√	PK3	该食谱的日期, 联合主键第三部分
MEAL_TYPE	VARCHAR2(20)		CK	餐型 (breakfast/lunch/dinner/snack)
NOTES	VARCHAR2(255)			特殊说明
ADDED_AT	TIMESTAMP	√	DF=SYSDATE	添加时间

1.8 视图设计方案

表 30: 视图设计方案

类别	视图名称	描述/用途	关键数据源
用户功能	USER__OVERVIEW (用户概览)	个人资料页展示 。聚合用户的基本信息、食谱数、粉丝数、关注数及平均评分	Users, Recipes, Followers, Ratings
	ACTIVE_USERS (活跃用户)	运营分析 。筛选一定天数内有发布、评分或评论行为的用户	Users, Activity Logs
食谱功能	RECIPE_DETAIL (食谱详情)	详情页展示 。封装食谱基本信息、作者信息及营养成分, 屏蔽已删除数据	Recipes, Users, Nutrition
	POPULAR_RECIPES (热门食谱)	推荐算法 。基于评价 (50%)、评价数 (30%)、浏览量 (20%) 计算热度分	Recipes, Ratings
	RECIPE_WITH_INGREDIENTS (食材清单)	详情页展示 。将食谱与食材、单位、过敏原表进行关联展示	Recipes, Ingredients, Units
	RECIPE_WITH_STEPS (烹饪步骤)	详情页展示 。按顺序提取烹饪步骤, 支持步骤导航 (上一/下一步)	Recipes, Cooking_Steps
	INGREDIENT_HEALTH_PROFILE (食材档案)	健康分析 。展示食材的分类、过敏原信息及在食谱中的使用频率	Ingredients, Allergens
	SAFE_RECIPES_FOR_USER (安全食谱)	个性化推荐 。自动过滤掉当前用户过敏原的食谱列表	Recipes, User_Allergies

表 30 – 续表

类别	视图名称	描述/用途	关键数据源
社交功能	USER_NETWORK (社交网络)	个人中心 。统计用户的社交圈数据（关注/粉丝/收藏/评论数）	Users, Followers, Saved_Recipes
	USER_CONTRIBUTION_SCORE (贡献评分)	排行榜计算 。基于食谱、粉丝、互动量加权计算用户的社区贡献分析	Users, Ratings, Comments
个性化功能	MEAL_PLAN_SUMMARY (计划摘要)	膳食管理 。统计膳食计划的天数、食谱数及覆盖时段	Meal_Plans, Meal_Plan_Entries
	RECOMMENDED_SHOPPING_LIST (购物清单)	工具功能 。合并膳食计划中所有食谱的食材，自动汇总同类食材数量	Meal_Plans, Ingredients
	ALL_SAVED_RECIPES (所有收藏食谱)	工具功能 。合并所有个人食谱收藏夹，作为总收藏夹展示	Meal_Plans, Ingredients
分析报表	RECIPE_QUALITY_METRICS (质量指标)	后台审核 。评估食谱质量（图片、步骤完整度、互动率）	Recipes, Comments, Steps
	MONTHLY_STATISTICS (月度统计)	数据整合 。按月统计新增食谱、活跃用户及互动总量	Recipes, Ratings, Comments

1.9 数据库安全方案的设计

1.9.1 数据库人员

根据系统运维与业务需求，划分以下五类数据库用户角色：

表 31: 数据库人员角色

用户角色	建议用户名	描述	核心权限/职责
应用用户	APP_USER	生产环境应用程序连接数据库使用的账户	拥有业务数据的增删改查权限，可执行业务存储过程
报表用户	REPORT_USER	用于 BI 报表、数据分析的只读账户	仅拥有业务数据的查询权限，严格限制对敏感字段的访问
管理员	DBA_ADMIN	数据库管理员 (DBA) 维护使用	拥有 DBA 最高权限，负责系统配置、用户管理等。
备份用户	BACKUP_USER	自动化备份脚本使用的账户	拥有导出数据库、读取所有数据的权限，用于灾难备份与恢复

表 31 – 续表

用户角色	建议用户名	描述	核心权限/职责
审计用户	AUDIT_USER	安全审计人员使用	查看数据库审计日志，监控异常访问行为。

1.9.2 权限分配

表 32: 权限分配方案

数据对象类别	具体对象示例	APP_USER	REPORT_USER	安全备注
核心业务表	RECIPES, RATINGS, COMMENTS	读写 (Select, Insert, Update, Delete)	只读 (Select)	公开业务数据，报表可分析。
用户敏感表	USERS	读写	受限只读	报表用户通过列级安全策略，不可见密码、手机号等隐私字段
私有数据表	SHOPPING_LISTS, MEAL_PLANS	读写	无权限	个人私有数据，仅应用端可操作，报表端无需访问。
基础字典表	INGREDIENTS, UNITS, TAGS	只读	只读	基础元数据，通常由后台管理，应用端仅引用。
业务逻辑	publish_recipe, rate_recipe (存储过程)	执行 (Execute)	无权	封装复杂业务逻辑，防止直接 SQL 注入。

2 实现部分

2.1 表格创建（可同时包括约束条件的创建）

为了节省正文篇幅，纸质版我们只展示部分表创建的代码，以及表的查询结果，完整代码我们将保存在电子版提交的附件。

2.1.1 核心表 USERS 与 RECIPES 的创建

USERS 表创建 以 USERS 表为例，我们为 user_id 设置了主键约束，确保用户唯一性；对 email 和 username 设置了 UNIQUE 约束，防止重复注册；并使用 DEFAULT 关键字为 join_date 设置默认值为当前时间。

```

1 -- 表1: USERS (用户表)
2 CREATE TABLE USERS (

```

```

3      user_id NUMBER(10) PRIMARY KEY,
4      username VARCHAR2(50) NOT NULL UNIQUE,
5      email VARCHAR2(100) NOT NULL UNIQUE,
6      password_hash VARCHAR2(255) NOT NULL,
7      first_name VARCHAR2(50),
8      last_name VARCHAR2(50),
9      bio VARCHAR2(500),
10     profile_image VARCHAR2(255),
11     join_date DATE DEFAULT SYSDATE NOT NULL,
12     last_login DATE,
13     account_status VARCHAR2(20) DEFAULT 'active' NOT NULL,
14     created_at TIMESTAMP DEFAULT SYSTIMESTAMP,
15     updated_at TIMESTAMP DEFAULT SYSTIMESTAMP,
16     CONSTRAINT ck_account_status CHECK (account_status IN
17     ('active', 'inactive', 'suspended'))
18 );

```

	USER_ID	EMAIL	BIO	ACCOUNT_STATUS
1	1	zhang.wei@email.com	专业美食摄影师，运营家常菜那些事公众号，分享地道家常菜做法	active
2	2	li.na.cooking@email.com	餐厅主厨5年，擅长川菜和湖南菜，热爱分享烹饪技巧	active
3	3	wangjun1990@email.com	上班族，热爱快手菜和健身餐，记录美食生活	active
4	4	fang.liu.home@email.com	全职妈妈，分享家常菜和儿童食谱，让孩子享受美食	active
5	5	chen.ming.chef@email.com	面食专家，自有面馆，专注于传统面食制作和创新	active
6	6	hui.xu.health@email.com	注册营养师，分享科学健康的食谱和营养知识	active
7	7	tao.zhou@email.com	美食爱好者，热爱尝试新菜谱，分享心得体会	active
8	8	guo.qiang.food@email.com	资深吃货，热爱测评各地特色菜，探寻美食故事	active
9	9	sun.yue.cook@email.com	大学生，想学做家常菜，与家人分享美食	active
10	10	jiang.lin.baker@email.com	西点烘焙师，分享烘焙技巧和精致甜品食谱	active

图 7: USERS 表查询截图

RECIPES 表创建 RECIPES 是另一个核心的表，我们为 recipe_id 设置了主键约束，确保食谱唯一性；对 user_id 设置了外键约束，关联到 USERS 表的 user_id，确保每个食谱都有作者。

```

1 CREATE TABLE RECIPES (
2     recipe_id NUMBER(10) PRIMARY KEY,
3     user_id NUMBER(10) NOT NULL,
4     recipe_name VARCHAR2(200) NOT NULL,
5     description VARCHAR2(1000),
6     cuisine_type VARCHAR2(50),
7     meal_type VARCHAR2(20),
8     difficulty_level VARCHAR2(20),
9     image_url VARCHAR2(255),
10    total_time NUMBER(5),
11    is_published VARCHAR2(1) DEFAULT 'Y' NOT NULL,
12    is_deleted VARCHAR2(1) DEFAULT 'N' NOT NULL,
13    created_at TIMESTAMP DEFAULT SYSTIMESTAMP,
14    updated_at TIMESTAMP DEFAULT SYSTIMESTAMP,
15    CONSTRAINT fk_recipes_user FOREIGN KEY (user_id)
16    REFERENCES USERS(user_id) ON DELETE CASCADE,

```

```

17  CONSTRAINT ck_meal_type CHECK (meal_type IN
18  ('breakfast', 'lunch', 'dinner', 'snack', 'dessert')),
19  CONSTRAINT ck_difficulty_level CHECK
20  (difficulty_level IN ('easy', 'medium', 'hard')),
21  CONSTRAINT ck_is_published CHECK
22  (is_published IN ('Y', 'N')),
23  CONSTRAINT ck_is_deleted CHECK
24  (is_deleted IN ('Y', 'N'))
25 );

```

	RECIPE_ID	USER_ID	RECIPE_NAME	CUISINE_TYPE	MEAL_TYPE	DIFFICULTY_LEVEL	IS_PUBLISHED
1	2	5	番茄鸡蛋面	家常	breakfast	easy	Y
2	3	1	红烧肉	经典	dinner	medium	Y
3	4	6	清蒸鱼	粤菜	dinner	easy	Y
4	5	2	麻婆豆腐	川菜	lunch	medium	Y
5	6	4	青菜炒蛋	家常	lunch	easy	Y
6	7	4	冬瓜排骨汤	家常	dinner	easy	Y
7	8	10	意大利面-番茄肉酱	意大利	lunch	easy	Y
8	9	6	凯撒沙拉	美式	lunch	easy	Y
9	10	8	烤鸡腿	西式	dinner	easy	Y
10	11	10	提拉米苏	意大利	dessert	easy	Y
11	12	8	日式味噌汤	日本	breakfast	easy	Y
12	13	9	寿司卷	日本	lunch	medium	Y
13	14	7	咖喱鸡饭	泰式	lunch	easy	Y
14	1	2	宫保鸡丁	川菜	lunch	medium	Y

图 8: RECIPES 表查询截图

2.1.2 转表 RECIPE_INGREDIENTS 创建

数据库设计中有许多表来自多对多关系，这里展示 RECIPE_INGREDIENTS 表的创建代码。该表关联了食谱和食材，包含了数量、单位等信息。

```

1  CREATE TABLE RECIPE_INGREDIENTS (
2      recipe_id NUMBER(10) NOT NULL,
3      ingredient_id NUMBER(10) NOT NULL,
4      unit_id NUMBER(10) NOT NULL,
5      quantity NUMBER(10,2) NOT NULL,
6      notes VARCHAR2(255),
7      added_at TIMESTAMP DEFAULT SYSTIMESTAMP,
8      PRIMARY KEY (recipe_id, ingredient_id),
9      CONSTRAINT fk_ri_recipe FOREIGN KEY (recipe_id)
10     REFERENCES RECIPES(recipe_id) ON DELETE CASCADE,
11     CONSTRAINT fk_ri_ingredient FOREIGN KEY (ingredient_id)
12     REFERENCES INGREDIENTS(ingredient_id),
13     CONSTRAINT fk_ri_unit FOREIGN KEY (unit_id)
14     REFERENCES UNITS(unit_id)
15 );

```

RECIPE_ID	INGREDIENT_ID	UNIT_ID	QUANTITY	NOTES	ADDED_AT
1	1	9	1	480 鸡肉切丁	2025-12-22T16:51:14.354
2	1	11	1	60 胡萝卜	2025-12-22T16:51:14.357
3	1	2	1	50 (null)	2025-12-22T16:51:14.358
4	1	20	1	60 (null)	2025-12-22T16:51:14.359
5	1	24	2	2 (null)	2025-12-22T16:51:14.360
6	2	1	1	300 新鲜蘑菇	2025-12-22T16:51:14.362
7	2	11	8	2 鸡蛋	2025-12-22T16:51:14.363
8	2	33	1	200 面条	2025-12-22T16:51:14.363
9	2	17	1	5 盐	2025-12-22T16:51:14.364

图 9: RECIPE_INGREDIENTS 表查询截图

2.1.3 有自引用的表

数据库设计中有所长表需要自引用关系，尤其是 COMMENTS 表和 FOLLOWERS 表。下面展示这两个表的创建代码。

COMMENTS 表创建

```

1  -- 表14: COMMENTS (评论表)
2  CREATE TABLE COMMENTS (
3      comment_id NUMBER(10) PRIMARY KEY,
4      user_id NUMBER(10) NOT NULL,
5      recipe_id NUMBER(10) NOT NULL,
6      comment_text VARCHAR2(1000) NOT NULL,
7      parent_comment_id NUMBER(10),
8      created_at TIMESTAMP DEFAULT SYSTIMESTAMP,
9      updated_at TIMESTAMP DEFAULT SYSTIMESTAMP,
10     is_deleted VARCHAR2(1) DEFAULT 'N' NOT NULL,
11     CONSTRAINT ck_comments_is_deleted CHECK (is_deleted IN ('Y', 'N')),
12     CONSTRAINT fk_comments_user FOREIGN KEY (user_id)
13     REFERENCES USERS(user_id) ON DELETE CASCADE,
14     CONSTRAINT fk_comments_recipe FOREIGN KEY (recipe_id)
15     REFERENCES RECIPES(recipe_id) ON DELETE CASCADE,
16     CONSTRAINT fk_comments_parent FOREIGN KEY (parent_comment_id)
17     REFERENCES COMMENTS(comment_id) ON DELETE CASCADE
18 );

```

	COMMENT_ID	USER_ID	RECIPE_ID	COMMENT_TEXT	PARENT_COMMENT_ID	CREATED_AT
1	1	3	1	请问花生可以用杏仁替代吗?	(null)	2025-12-18T6
2	2	2	1	可以的, 杏仁的香气也很不错, 用量可以少一点	1	2025-12-19T6
3	3	4	2	有没有其他配菜的建议?	(null)	2025-12-17T6
4	4	3	3	多久可以吃完一份红烧肉?	(null)	2025-12-16T6
5	5	1	3	一般两三天内吃完比较好, 可以冷藏保存	4	2025-12-17T6
6	6	6	4	蒸鱼用什么样的火候最好?	(null)	2025-12-15T6
7	7	2	4	中火蒸12-15分钟就够了, 过度蒸会变老	6	2025-12-16T6
8	8	9	13	寿司卷怎样防止散开?	(null)	2025-12-21T6
9	9	5	13	米要均匀摊平, 海苔要新鲜, 切刀要锋利并蘸水	8	2025-12-22T6
10	10	4	6	小孩喜欢吃这个, 简直是完美的儿童菜	(null)	2025-12-20T6
11	11	3	8	这个番茄肉酱可以冷冻保存吗?	(null)	2025-12-21T6
12	12	10	8	可以的, 冷冻可以保存2-3周, 用时解冻即可	11	2025-12-22T6

图 10: COMMENTS 表查询截图

比如这里的 id 为 2 的评论就是对 id 为 1 的评论的回复。

FOLLOWERS 表创建

```
1 -- 表17: FOLLOWERS (用户关注关系表 - 自引用多对多 - 联合主键)
2 CREATE TABLE FOLLOWERS (
3     user_id NUMBER(10) NOT NULL,
4     follower_user_id NUMBER(10) NOT NULL,
5     followed_at TIMESTAMP DEFAULT SYSTIMESTAMP,
6     PRIMARY KEY (user_id, follower_user_id),
7     CONSTRAINT fk_followers_user FOREIGN KEY (user_id)
8     REFERENCES USERS(user_id) ON DELETE CASCADE,
9     CONSTRAINT fk_followers_follower FOREIGN KEY (follower_user_id)
10    REFERENCES USERS(user_id) ON DELETE CASCADE,
11    CONSTRAINT ck_not_self_follow CHECK (user_id != follower_user_id)
12 );
```

	USER_ID	FOLLOWER_USER_ID	FOLLOWED_AT
1	1	3	2025-11-22T00:00
2	1	4	2025-11-27T00:00
3	1	7	2025-12-02T00:00
4	2	3	2025-11-24T00:00
5	2	9	2025-12-07T00:00
6	6	3	2025-11-17T00:00
7	6	4	2025-11-30T00:00
8	8	7	2025-12-12T00:00

图 11: FOLLOWERS 表查询截图

Followers 的每条 record 只表示一个用户关注另一个用户。

2.2 数据录入

数据导入部分，先为需要自增主键的表创建序列，实现自动生成主键，接下来使用插入模拟数据。

在实验的过程中，发现先创建序列，再插入数据，序列居然会自动先往前“走一步”，与 AI 互动后发现既不是使用了触发器的问题（有空学一下），也不是 Cache 的问题。然而把表格数据删去，把序列 drop 掉后，重新创建序列，插数据，居然就不会出现这个问题了。所以，在写插入数据的脚本的时候，我只好把需要使用序列的表创建两遍。最后的数据还是可以正常使用的。

	LABEL	CNT
1	用户	10
2	食材	35
3	单位	12
4	过敏原	8
5	标签	12
6	食谱	14
7	烹饪步骤	58
8	食谱食材	49
9	营养信息	14
10	食谱标签	24
11	食材过敏原	7
12	评价	20
13	评论	12
14	收藏	14
15	关注	8
16	用户过敏原	5
17	食谱清单	3
18	清单食谱	12
19	购物清单	2
20	清单项目	15
21	膳食计划	0
22	计划条目	0

图 12: 各表基数统计截图

2.3 视图创建和使用

2.3.1 视图创建概览

我们小组主要聚焦与用户使用相关的绝大多数视图，因为可能与用户相关的话，业务逻辑可以更加明显地看到，更好理解与设计。

我们尽可能完成了前面设计方案中的大部分视图，以下是我们创建的视图列表：

2.3.2 用户相关视图

USER_OVERVIEW 视图创建 用户相关视图主要用在用户个人主页展示用户的主要信息。其中，用户的食谱总数于用户的关注人数还有粉丝人数是派生属性，需要通过聚合函数计算得到。

```

1  -- 视图：USER_OVERVIEW (用户概览视图)
2  CREATE OR REPLACE VIEW USER_OVERVIEW AS
3  SELECT
4      u.user_id,
5      u.username,
6      u.account_status,
7      COUNT(DISTINCT r.recipe_id) AS recipe_count,
8      COUNT(DISTINCT f.follower_user_id) AS follower_count,
9      COUNT(DISTINCT f2.user_id) AS following_count,
10     u.profile_image,
11     ROUND(AVG(rt.rating_value), 2) AS avg_recipe_rating,
12     u.bio
13 FROM USERS u
14 LEFT JOIN RECIPES r ON u.user_id = r.user_id AND r.is_deleted = 'N'

```

```

15 LEFT JOIN FOLLOWERS f ON u.user_id = f.user_id
16 LEFT JOIN FOLLOWERS f2 ON u.user_id = f2.follower_user_id
17 LEFT JOIN RATINGS rt ON r.recipe_id = rt.recipe_id
18 GROUP BY u.user_id, u.username, u.first_name, u.last_name, u.profile_image,
19          u.bio, u.join_date, u.account_status;

```

	USER_ID	USERNAME	ACCOUNT_STATUS	RECIPE_COUNT	FOLLOWER_COUNT	FOLLOWING_COUNT	PROFILE_IMAGE	AVG_RECIPE_RATING	BIO
1	1	zhang_wei	active	1	3	0	(null)	4.65	专业美食摄
2	2	li_na	active	2	2	0	(null)	4.3	餐厅主厨5年
3	3	wang_jun	active	0	0	3	(null)	(null)	上班族, 热爱
4	4	liu_fang	active	2	0	2	(null)	4.5	全职妈妈, 热爱
5	5	chen_ming	active	1	0	0	(null)	4.75	面食专家, 热爱
6	6	xu_hui	active	2	2	0	(null)	4.5	注册营养师
7	7	zhou_tao	active	1	0	2	(null)	4.5	美食爱好者
8	8	guo_qiang	active	2	1	0	(null)	4.25	资深吃货, 热爱
9	9	sun_yue	active	1	0	1	(null)	4.5	大学生, 热爱
10	10	jiang_lin	active	2	0	0	(null)	4.75	西点烘焙师

图 13: 用户基本信息视图截图

USER_SAFE_RECIPES 视图创建 这个视图主要是为了给用户推荐不含其过敏原的食谱，提升用户体验和安全性。也必须是只读视图，防止误操作给用户推荐过敏食谱。

```

1 CREATE OR REPLACE VIEW USER_SAFE_RECIPES AS
2 SELECT
3     u.user_id, r.recipe_id, r.recipe_name, r.average_rating
4 FROM USERS u
5 CROSS JOIN RECIPES r
6 WHERE r.is_published = 'Y'
7     AND r.is_deleted = 'N'
8     AND NOT EXISTS (
9         SELECT 1
10        FROM RECIPE_INGREDIENTS ri
11        JOIN INGREDIENT_ALLERGENS ia ON ri.ingredient_id = ia.ingredient_id
12        JOIN USER_ALLERGIES ua ON ia.allergen_id = ua.allergen_id
13        WHERE ri.recipe_id = r.recipe_id
14              AND ua.user_id = u.user_id
15     )
16 with READ ONLY;

```

	USER_ID	RECIPE_ID	RECIPE_NAME	AVERAGE_RATING
1	1	1	宫保鸡丁	4.6
2	1	2	番茄鸡蛋面	4.75
3	1	3	红烧肉	4.8
4	1	4	清蒸鱼	4.5
5	1	5	麻婆豆腐	4.4
6	1	6	青菜炒蛋	4.7

图 14: 用户安全食谱视图截图

2.3.3 食谱相关视图

食谱视图主要用于食谱详情页的展示，第一层是只有食谱基本信息的视图，第二层是由详细烹饪步骤的视图。

RECIPE_WITH_INGREDIENT 视图创建 RECIPE_WITH_INGREDIENT 主要用于在食谱的详情页中显示食谱的原料清单，对于浏览者来说是只读的。

这里也遇到了一个问题，就是不知道如何聚合显示一个原料的所有别名，如果每个别名都要一行的话，这个视图就有点太大了。

```
1 CREATE OR REPLACE VIEW RECIPE_WITH_INGREDIENTS AS
2 SELECT
3     r.recipe_id,
4     r.recipe_name,
5     r.servings,
6     ri.ingredient_id,
7     i.ingredient_name,
8     i.category AS ingredient_category,
9     ri.quantity,
10    u.unit_name,
11    u.abbreviation,
12    ri.notes
13 FROM RECIPES r
14 JOIN RECIPE_INGREDIENTS ri ON r.recipe_id = ri.recipe_id
15 JOIN INGREDIENTS i ON ri.ingredient_id = i.ingredient_id
16 JOIN UNITS u ON ri.unit_id = u.unit_id
17 WHERE r.is_deleted = 'N',
18 WITH READ ONLY;
```

	RECIPE_ID	RECIPE_NAME	SERVINGS	INGREDIENT_ID	INGREDIENT_NAME	INGREDIENT_CATEGORY
1	1	宫保鸡丁	3	9	鸡肉	肉类
2	1	宫保鸡丁	3	11	鸡蛋	肉类
3	1	宫保鸡丁	3	2	洋葱	蔬菜
4	1	宫保鸡丁	3	20	醋	调味料
5	1	宫保鸡丁	3	24	料酒	调味料
6	2	番茄鸡蛋面	2	1	番茄	蔬菜
7	2	番茄鸡蛋面	2	11	鸡蛋	肉类
8	2	番茄鸡蛋面	2	33	蜂蜜	其他
9	2	番茄鸡蛋面	2	17	盐	调味料

图 15: 食谱及其原料视图截图

RECIPE_WITH_STEPS 视图创建 该视图用于进一步展示食谱的烹饪步骤，方便网页的浏览者查看，也应该是只读视图。

```
1 CREATE OR REPLACE VIEW RECIPE_WITH_STEPS AS
2 SELECT
3     r.recipe_id,
4     r.recipe_name,
```

```

5      cs.step_number,
6      cs.instruction,
7      cs.time_required,
8      cs.image_url,
9      LAG(cs.step_number) OVER (PARTITION BY r.recipe_id ORDER BY cs.step_number)
10     AS previous_step,
11      LEAD(cs.step_number) OVER (PARTITION BY r.recipe_id ORDER BY cs.step_number)
12     AS next_step
13 FROM RECIPES r
14 LEFT JOIN COOKING_STEPS cs ON r.recipe_id = cs.recipe_id
15 WHERE r.is_deleted = 'N'
16 ORDER BY r.recipe_id, cs.step_number;

```

RECIPE_NAME	STEP_NUMBER	INSTRUCTION	TIME_REQUIRED
宫保鸡丁	1	鸡肉切丁（约1厘米见方），花生米炒香备用，干辣椒切段	5
宫保鸡丁	2	鸡丁用盐、料酒、淀粉腌制10分钟，锁住水分	3
宫保鸡丁	3	锅中油热，放入鸡丁快速翻炒至变白，盛起备用	5
宫保鸡丁	4	锅中再下油，炒香干辣椒和葱段，倒入鸡丁翻炒	3
宫保鸡丁	5	加入酱油、糖、醋调味，最后加入花生米，炒匀即可	2
番茄鸡蛋面	1	番茄洗净切块，鸡蛋打散备用	3
番茄鸡蛋面	2	锅中油热，倒入蛋液快速炒散盛起	3
番茄鸡蛋面	3	同一锅加油炒番茄块，出汁后加入清汤烧开	4
番茄鸡蛋面	4	下面条煮至半熟，加入鸡蛋和调料，煮2分钟即可	3

图 16: 食谱及其烹饪步骤视图截图

2.3.4 社交互动视图

AllRecipe 还有社交功能，尤其是在食谱下方的用户评论区，所以我们设计了一些社交互动相关的视图。

RECIPE_COMMENTS_DETAIL 视图创建

```

1 CREATE OR REPLACE VIEW RECIPE_COMMENTS_DETAIL AS
2 SELECT
3     r.recipe_id,
4     r.recipe_name,
5     c.comment_id,
6     u.user_id,
7     u.username AS commenter_name,
8     u.profile_image AS commenter_avatar,
9     c.comment_text,
10    c.parent_comment_id,
11    c.created_at,
12    (SELECT COUNT(*) FROM COMMENT_HELPFULNESS ch
13     WHERE ch.comment_id = c.comment_id) AS helpful_count
14 FROM COMMENTS c
15 JOIN RECIPES r ON c.recipe_id = r.recipe_id
16 JOIN USERS u ON c.user_id = u.user_id

```

```

17 WHERE c.is_deleted = 'N' AND r.is_deleted = 'N'
18 WITH READ ONLY;

```

	RECIPE_NAME	COMMENT_ID	PARENT_COMMENT_ID	COMMENTS_NAME	COMMENT_TEXT	CREATED_AT	RECIPE_ID
1	红烧肉	5	4	zhang_wei	一般两三天内吃完比较好，可以冷藏保存	2025-12-17T00:00	3
2	宫保鸡丁	2	1	li_na	可以的，杏仁的香气也很不错，用量可以少一点	2025-12-19T00:00	1
3	清蒸鱼	7	6	li_na	中火蒸12-15分钟就够了，过度蒸会变老	2025-12-16T00:00	4
4	宫保鸡丁	1	(null)	wang_jun	请问花生可以用杏仁替代吗？	2025-12-18T00:00	1
5	红烧肉	4	(null)	wang_jun	多久可以吃完一份红烧肉？	2025-12-16T00:00	3
6	意大利面-番茄肉酱	11	(null)	wang_jun	这个番茄肉酱可以冷冻保存吗？	2025-12-21T00:00	8
7	番茄鸡蛋面	3	(null)	liu_fang	有没有其他配菜的建议？	2025-12-17T00:00	2
8	青菜炒蛋	10	(null)	liu_fang	小孩喜欢吃这个，简直是完美的儿童菜	2025-12-20T00:00	6
9	寿司卷	9	8	chen_ming	米要均匀摊平，海苔要新鲜，切刀要锋利并蘸水	2025-12-22T00:00	13
10	清蒸鱼	6	(null)	xu_hui	蒸鱼用什么样的火候最好？	2025-12-15T00:00	4
11	寿司卷	8	(null)	sun_yue	寿司卷怎样防止散开？	2025-12-21T00:00	13
12	意大利面-番茄肉酱	12	11	jiang_lin	可以的，冷冻可以保存2-3周，用时解冻即可	2025-12-22T00:00	8

图 17: 食谱及其用户评论视图截图

2.3.5 分析报表视图

这一部分的视图主要用于后台管理和数据分析，帮助运营团队了解平台的使用情况和用户行为。

MONTHLY_STATISTICS 视图创建

```

1 SELECT
2     TRUNC(r.created_at, 'MM') AS month,
3     COUNT(DISTINCT r.recipe_id) AS new_recipes,
4     COUNT(DISTINCT r.user_id) AS active_creators,
5     COUNT(DISTINCT rt.rating_id) AS total_ratings,
6     COUNT(DISTINCT c.comment_id) AS total_comments,
7     ROUND(AVG(r.average_rating), 2) AS avg_rating
8 FROM RECIPES r
9 LEFT JOIN RATINGS rt ON r.created_at = TRUNC(rt.rating_date, 'MM')
10 LEFT JOIN COMMENTS c ON r.created_at = TRUNC(c.created_at, 'MM')
11 WHERE r.is_deleted = 'N'
12 GROUP BY TRUNC(r.created_at, 'MM')
13 ORDER BY month DESC;

```

2.4 常见操作的实例演示

我们接下来将复现一个用户在网站上可能会做的比较多的操作。并且按照增删改查分类。

2.4.1 插入操作

由于我们的视图中绝大多数都使用了聚合函数或者 WITH READ ONLY 选项，所以插入操作只能在表上进行。

新用户注册 下面以 USERS 表为例，展示插入操作。

```

1 -- 创建新用户
2 INSERT INTO USERS (
3     user_id, username, email, password_hash, first_name, last_name,
4     bio, join_date, account_status, created_at, updated_at

```

```

5 )
6 VALUES (
7     seq_users.NEXTVAL, 'new_user', 'newuser@example.com', 'hashed_password_123',
8     'New', 'User', '我是一位美食爱好者', SYSDATE, 'active', SYSTIMESTAMP, SYSTIMESTAMP
9 );
10 COMMIT;

```

	USER_ID	USERNAME	EMAIL	PASSWORD_HASH	FIRST_NAME	LAST_NAME	BIO
1	1	zhang_wei	zhang.wei@email.com	pwd_hash_zw123	Zhang	Wei	专业美
2	2	li_na	li.na.cooking@email.com	pwd_hash_ln456	Li	Na	餐厅主
3	3	wang_jun	wangjun1990@email.com	pwd_hash_wj789	Wang	Jun	上班族
4	4	liu_fang	fang.liu.home@email.com	pwd_hash_lf111	Liu	Fang	全职妈
5	5	chen_ming	chen.ming.chef@email.com	pwd_hash_cm222	Chen	Ming	面食专
6	6	xu_hui	hui.xu.health@email.com	pwd_hash_xh333	Xu	Hui	注册营
7	7	zhou_tao	tao.zhou@email.com	pwd_hash_zt444	Zhou	Tao	美食爱
8	8	guo_qiang	guo.qiang.food@email.com	pwd_hash_gq555	Guo	Qiang	资深吃
9	9	sun_yue	sun.yue.cook@email.com	pwd_hash_sy666	Sun	Yue	大学生
10	10	jiang_lin	jiang.lin.baker@email.com	pwd_hash_jl777	Jiang	Lin	西点烘
11	11	nwf	202411089@sufe.edu.cn	IloveDBLearning	Jane	Doe	新手烹

图 18: 新用户注册截图

实际上这个操作一般在网站上注册用户时由应用程序完成。

新食谱发布

```

1 -- 第一步：插入食谱主表
2 INSERT INTO RECIPES (
3     recipe_id, user_id, recipe_name, description, cuisine_type,
4     meal_type, difficulty_level, prep_time, cook_time, servings,
5     is_published, is_deleted, created_at, updated_at
6 )
7 VALUES (
8     seq_recipes.NEXTVAL, 11, '番茄鸡蛋汤',
9     '清汤营养，简单快手，适合全年齡段食用',
10    '家常', 'lunch', 'easy', 5, 10, 2,
11    'Y', 'N', SYSTIMESTAMP, SYSTIMESTAMP
12 );
13
14 select * from RECIPES where recipe_name='番茄鸡蛋汤';
15
16 -- 第二步：插入食谱食材（外键依赖 RECIPES、INGREDIENTS、UNITS）
17 INSERT INTO RECIPE_INGREDIENTS (
18     recipe_id, ingredient_id, unit_id, quantity, notes
19 ) VALUES (15, 1, 1, 300, '新鲜番茄，切块');
20
21 INSERT INTO RECIPE_INGREDIENTS (
22     recipe_id, ingredient_id, unit_id, quantity, notes
23 ) VALUES (15, 11, 8, 2, '打散后倒入汤中');
24
25 select * from RECIPE_INGREDIENTS where recipe_id=15;

```

```

26
27 -- 第三步：插入烹饪步骤
28 INSERT INTO COOKING_STEPS (
29     step_id, recipe_id, step_number, instruction, time_required
30 ) VALUES (seq_cooking_steps.NEXTVAL,15,1,'番茄洗净切块，鸡蛋打碎备用',5);
31
32 INSERT INTO COOKING_STEPS (
33     step_id, recipe_id, step_number, instruction, time_required
34 ) VALUES (seq_cooking_steps.NEXTVAL,15,2,
35 '烧开一锅清水，加入番茄块，煮 5 分钟至番茄软烂',5);
36
37 INSERT INTO COOKING_STEPS (
38     step_id, recipe_id, step_number, instruction, time_required
39 ) VALUES (seq_cooking_steps.NEXTVAL,15,3,
40 '倒入打散的鸡蛋，边倒边搅拌形成蛋花，加盐调味，完成',5);

```

	RECIPE_ID	USER_ID	RECIPE_NAME	DESCRIPTION	CUISINE_TYPE	MEAL_TYPE	DIFFICULTY_LEVEL	PREP_TIME	COOK
1	15	11	番茄鸡蛋汤	清汤营养，简单快手，适合全年龄段食用	家常	lunch	easy		5

图 19: 创建新食谱截图

	RECIPE_ID	INGREDIENT_ID	UNIT_ID	QUANTITY	NOTES	ADDED_AT
1	15	1	1	300	新鲜番茄，切块	2025-12-24T10:40:09.768
2	15	11	8	2	打散后倒入汤中	2025-12-24T10:41:05.938

图 20: 加入食材截图

	STEP_ID	RECIPE_ID	STEP_NUMBER	INSTRUCTION	TIME_REQUIRED	IMAGE_URL
1	59	15	1	番茄洗净切块，鸡蛋打碎备用	5	(null)
2	60	15	2	烧开一锅清水，加入番茄块，煮 5 分钟至番茄软烂	5	(null)
3	61	15	3	倒入打散的鸡蛋，边倒边搅拌形成蛋花，加盐调味，完成	5	(null)

图 21: 更新烹饪步骤截图

2.4.2 查询操作

查询操作可以使用我们创建的诸多视图来简化复杂的查询。

用户查看某一食谱 查询食谱一共要显示这个食谱的原料、食谱的烹饪步骤以及食谱的用户评论。

```

1 select * from RECIPE_WITH_INGREDIENTS where recipe_id=1;
2 select * from RECIPE_WITH_STEPS where recipe_id=1;
3 select * from RECIPE_COMMENTS_DETAIL where recipe_id=1;

```

	RECIPE_ID	RECIPE_NAME	SERVINGS	INGREDIENT_ID	INGREDIENT_NAME	INGREDIENT_CATEGORY	QUANTITY	UNIT_NAME	ABBREVIATION	N
1	1	宫保鸡丁	3	11	鸡蛋	肉类	60	克	g	片
2	1	宫保鸡丁	3	11	鸡蛋	肉类	60	克	g	片
3	1	宫保鸡丁	3	20	醋	调味料	60	克	g	(
4	1	宫保鸡丁	3	2	洋葱	蔬菜	50	克	g	(
5	1	宫保鸡丁	3	24	料酒	调味料	2	千克	kg	(
6	1	宫保鸡丁	3	9	鸡肉	肉类	400	克	g	两

图 22: 网站显示食谱的原料

	RECIPE_ID	RECIPE_NAME	STEP_NUMBER	INSTRUCTION	TIME_REQUIRED	IMAGE_URL	PREVIOUS_STEP	NEXT_S
1	1	宫保鸡丁	1	鸡肉切丁（约1厘米见方），花生米炒香备用，干辣椒切段	5	(null)	(null)	
2	1	宫保鸡丁	2	鸡丁用盐、料酒、淀粉腌制10分钟，锁住水分	3	(null)	1	
3	1	宫保鸡丁	3	锅中油热，放入鸡丁快速翻炒至变白，盛起备用	5	(null)	2	
4	1	宫保鸡丁	4	锅中再下油，炒香干辣椒和葱段，倒入鸡丁翻炒	3	(null)	3	
5	1	宫保鸡丁	5	加入酱油、糖、醋调味，最后加入花生米，炒匀即可	2	(null)	4	

图 23: 网站显示食谱的烹饪步骤

	RECIPE_ID	RECIPE_NAME	COMMENT_ID	USER_ID	COMMENTER_NAME	COMMENTER_AVATAR	COMMENT_TEXT	PARENT_COMME
1	1	宫保鸡丁	1	3	wang_jun	(null)	请问花生可以用杏仁替代吗?	
2	1	宫保鸡丁	2	2	li_na	(null)	可以的，杏仁的香气也很不错，用量可以少一点	

图 24: 网站显示食谱的用户评论

用户根据食材查询食谱 食谱网站有根据食材搜索的功能。一般在查询的过程中，我们希望用户输入某种食材，系统能够返回包含该食材或其替代品的所有食谱列表。所以，我们设计了如下的 SQL 查询语句，查询所有含有“猪肉”的食谱，另外应该附带有“五花肉”：

```

1 SELECT DISTINCT
2     r.recipe_name,
3     i.ingredient_name
4 FROM RECIPES r
5 JOIN RECIPE_INGREDIENTS ri ON r.recipe_id = ri.recipe_id
6 JOIN INGREDIENTS i ON ri.ingredient_id = i.ingredient_id
7 WHERE i.ingredient_id IN (
8     SELECT ingredient_id
9     FROM INGREDIENTS
10    WHERE ingredient_name = '猪肉'
11    UNION
12    SELECT sub.substitute_ingredient_id
13    FROM INGREDIENT_SUBSTITUTIONS sub
14    JOIN INGREDIENTS my_inventory ON
15    sub.original_ingredient_id = my_inventory.ingredient_id
16    WHERE my_inventory.ingredient_name = '猪肉'
17 )
18 AND r.is_published = 'Y'
19 AND r.is_deleted = 'N';

```

	RECIPE_NAME	INGREDIENT_NAME
1	意大利面-番茄肉酱	五花肉
2	红烧肉	猪肉

图 25: 网站显示用户查询结果

用户查看食谱排行榜 这里面我们也是设计了一个视图，但没在前面展示，其实是可以直接调用视图查询的。另外这里的推荐算法实际应用中可以使用更加复杂的模型来实现，我们使用了一个简单的加权评分模型。

```

1 SELECT
2     r.recipe_id,r.recipe_name,r.cuisine_type,r.average_rating,
3     r.rating_count,r.view_count,u.username,
4     ROUND(
5         r.average_rating * 0.5 +
6         LEAST(r.rating_count / 100.0, 5) * 0.3 +
7         LEAST(r.view_count / 10000.0, 5) * 0.2
8     , 2) AS popularity_score
9 FROM RECIPES r
10 JOIN USERS u ON r.user_id = u.user_id
11 WHERE r.is_published = 'Y'
12     AND r.is_deleted = 'N'
13     AND r.created_at > SYSDATE - 180  -- 过去6个月
14 ORDER BY popularity_score DESC;

```

	RECIPE_ID	RECIPE_NAME	AVERAGE_RATING
1	3	红烧肉	4.8
2	2	番茄鸡蛋面	4.75
3	11	提拉米苏	4.75
4	6	青菜炒蛋	4.7

图 26: 网站显示食谱排行榜

2.4.3 更新操作

用户更新个人资料

```

1 UPDATE USERS
2 SET
3     bio = '新手烹饪爱好者，分享家常菜做法',
4     profile_image = 'https://avatars.githubusercontent.com/u/145841814?v=4',
5     updated_at = SYSTIMESTAMP
6 WHERE user_id = 11;

```

	USER_ID	BIO	PROFILE_IMAGE
1	11	新手烹饪爱好者，分享家常菜做法	https://avatars.githubusercontent.com/u/145841814?v=4

图 27: 网站显示用户更新后的个人资料

用户发布食谱

```

1 UPDATE RECIPES
2 SET
3     is_published = 'Y',
4     updated_at = SYSTIMESTAMP
5 WHERE recipe_id = 5;

```

	RECIPE_ID	USER_ID	IS_PUBLISHED
1	5	2	Y

图 28: 网站显示用户发布后的食谱

系统自动更新 我们考虑到数据库中有许多派生属性，其中有些属性需要定期更新，从而提高系统性能，避免每次调用系统都要做复杂的查询，比如用户的好友总数和食谱总数。但是，食谱的准备时间、烹饪时间、总时间、营养等派生属性我们认为不需要定期更新，每次查询时计算即可，因为这些属性不会频繁变化。

```

1 UPDATE USERS
2 SET
3     total_recipes = (
4         SELECT COUNT(*)
5         FROM RECIPES
6         WHERE user_id = USERS.user_id
7             AND is_deleted = 'N'
8     ),
9     updated_at = SYSTIMESTAMP
10 WHERE account_status = 'active';

```

```

1 UPDATE RECIPES r
2 SET
3     average_rating = (
4         SELECT ROUND(AVG(rating_value), 2)
5         FROM RATINGS WHERE recipe_id = r.recipe_id
6     ),
7     rating_count = (
8         SELECT COUNT(*) FROM RATINGS
9         WHERE recipe_id = r.recipe_id
10    ), updated_at = SYSTIMESTAMP
11 WHERE recipe_id IN (

```

```

12 SELECT DISTINCT recipe_id FROM RATINGS
13 WHERE rating_date >= SYSDATE - 1
14 );

```

2.4.4 删除操作

我们了解到，一个网站的删除操作分物理删除和逻辑删除，其实逻辑删除应该属于更新操作的一种，但为了突出其重要性，我们单独列出来。

逻辑删除 只需要将 is_deleted 字段更新为'Y' 即可。

```

1 UPDATE RECIPES
2 SET
3 is_deleted = 'Y', updated_at = SYSTIMESTAMP
4 WHERE recipe_id = 10;

```

	RECIPE_ID	IS_DELETED
1	10	Y

图 29: 用户删除食谱

物理删除 物理删除一般由系统定期执行的清理任务完成，删除那些逻辑删除且超过一定时间未恢复的数据。

```

1 DELETE FROM RECIPES
2 WHERE is_deleted = 'Y'
3 AND SYSDATE - updated_at > 90;

```

2.5 数据库安全的实现

2.5.1 数据库用户角色划分

```

1 -- 创建不同权限等级的数据库用户
2
3 -- 1. 应用用户 (APP_USER) : 生产应用使用
4 CREATE USER app_user IDENTIFIED BY "SecureP@ss123";
5 GRANT CONNECT, RESOURCE TO app_user;
6
7 -- 2. 只读报表用户 (REPORT_USER) : 用于报表和分析
8 CREATE USER report_user IDENTIFIED BY "ReportP@ss456";
9 GRANT CONNECT TO report_user;
10
11 -- 3. 数据库管理员 (DBA_ADMIN) : DBA使用
12 CREATE USER dba_admin IDENTIFIED BY "AdminP@ss789";

```

```

13 GRANT DBA TO dba_admin;
14
15 -- 4. 备份用户 (BACKUP_USER) : 备份和恢复
16 CREATE USER backup_user IDENTIFIED BY "BackupP@ss000";
17 GRANT CREATE SESSION, EXPORT_FULL_DATABASE TO backup_user;
18
19 -- 5. 审计用户 (AUDIT_USER) : 审计日志查看
20 CREATE USER audit_user IDENTIFIED BY "AuditP@ss111";
21 GRANT CONNECT TO audit_user;

```

2.5.2 精细化权限分配

```

1 -- 为 APP_USER 分配表级权限
2 GRANT SELECT, INSERT, UPDATE, DELETE ON USERS TO app_user;
3 GRANT SELECT, INSERT, UPDATE, DELETE ON RECIPES TO app_user;
4 GRANT SELECT, INSERT, UPDATE, DELETE ON RATINGS TO app_user;
5 GRANT SELECT, INSERT, UPDATE, DELETE ON COMMENTS TO app_user;
6 GRANT SELECT, INSERT, UPDATE, DELETE ON FOLLOWERS TO app_user;
7 GRANT SELECT, INSERT, UPDATE, DELETE ON SAVED_RECIPES TO app_user;
8 GRANT SELECT, INSERT, UPDATE, DELETE ON SHOPPING_LISTS TO app_user;
9 GRANT SELECT, INSERT, UPDATE, DELETE ON SHOPPING_LIST_ITEMS TO app_user;
10 GRANT SELECT, INSERT, UPDATE, DELETE ON MEAL_PLANS TO app_user;
11 GRANT SELECT, INSERT, UPDATE, DELETE ON MEAL_PLAN_ENTRIES TO app_user;
12
13 -- 只读权限
14 GRANT SELECT ON INGREDIENTS TO app_user;
15 GRANT SELECT ON UNITS TO app_user;
16 GRANT SELECT ON TAGS TO app_user;
17 GRANT SELECT ON ALLERGENS TO app_user;
18
19 -- 为 REPORT_USER 分配只读权限
20 GRANT SELECT ON RECIPES TO report_user;
21 GRANT SELECT ON USERS TO report_user;
22 GRANT SELECT ON RATINGS TO report_user;
23 GRANT SELECT ON COMMENTS TO report_user;
24 GRANT SELECT ON FOLLOWERS TO report_user;
25 GRANT SELECT ON ALL VIEWS TO report_user;
26
27 -- 执行存储过程权限
28 GRANT EXECUTE ON publish_recipe TO app_user;
29 GRANT EXECUTE ON rate_recipe TO app_user;
30 GRANT EXECUTE ON save_recipe TO app_user;

```

3 总结

3.1 组员分工

小组分工, 我们小组之间分工比较均匀, 大家基本上都参与到了数据库设计与实践的各个环节。工作侧重略有不同:

表 33: 组员分工表

姓名	学号	主要工作
倪伟丰	2024110089	组长统筹安排工作, 物理模型设计, 数据库实现, 撰写报告
王宇阳	2024110090	语义建模, ER 图绘制, 物理模型设计, 撰写报告
史家和	2024110469	背景与业务调查, 查找数据, 数据库实现, 撰写报告

3.2 难点与不足分析

总体而言, 我们设计了一个对于比较庞大的数据库。我们的希望是能设计一个与 AllRecipe 网站业务尽可能接近的数据库, 从而更好地为其做好后端的支持。另外, 我们在 AI 的辅助下, 发现了在实际的应用场景下, 实体的属性是非常多的, 尤其是要支持数据库在网站上运行, 需要增加很多时间戳属性, 也学到了逻辑删除这种优雅的处理方式。据说有些小组还开发了前端的 Demo, 我们这里实在是卷不动了, 而且开发了 Demo 可能会在 AllRecipe 面前黯然失色。以下我们分板块总结一下设计和实践过程中遇到的问题, 以及我们认为还有改进空间的地方。

3.2.1 难点

概念模型设计中的挑战 购物车模型和评论模型是语义建模、ER 图绘制、转逻辑表以及物理表设计中比较棘手的一部分。我们不得不再三斟酌在两种情况下使用课上说的哪一种方式更加符合网页的业务逻辑, 更加便于规范化;

ER 图绘制 在绘制的过程中发现, 我们设计的很多实体的属性实在是太多了, 一张图上很难优雅地容纳下, 所以做了略微的删减, 但是最核心与最关键的部分都保留下来了。

设计规范与性能的平衡 我们在物理表中有很多属性是派生属性, 这本来是不应该放进来的。但是考虑到系统的性能, 以及想给我们自己偷个懒在后面写查询代码的时候省点里, 不用大量子查询和连接, 还是把派生属性放进去了。我们在想能否使用触发器或者函数帮助我们。

表太多导致的问题 由于表太多, 模型太复杂, 所以导致我们实现的过程中举步维艰。但是我们尽可能一步步给出模拟数据, 并且一步步地插入到表格中, 通过不断的试错, 最终实现了数据库的部分功能。

操作过程的问题 我们在查询 Recipe_With_Ingredients 视图的过程中, 想把一个食材的所有替代名称都显示出来, 但是我们没能使用聚合函数或者窗口函数解决这个问题。

3.2.2 不足

子类 and 超类没有使用 我们在设计 ER 模型的时候没有使用子类和超类，其实，我们发现了 AllRecipe 网站对用户类别也做了区分，分为了普通用户、专业厨师与美食博主。这个地方非常适合使用子类和超类。但是当我们发现的时候已经开始设计物理模型了，想想还是不要重新再来了，有些遗憾。老师要不就当我们将就使用了吧。

ER 图的美中不足 有一个实体 UNITS，表示食材的单位，由于和过多的实体有关系，所以没画进去。

数据库功能测试不完整 我们设计了很复杂的数据库，但是我们最后只实现了一部分，而且数据量很少与真实的数据库实践还是有很大的差距。

团队协作与版本控制 我们的报告与实践过程 3 人一同参与，难免有想法不一致或者进度不同步的情况，各种模型也是不断地被修改调整。在报告中难免有一些问题，以及设计部分与实践部分不一致的情况。

3.3 收获或体会及建议

这次数据库课程设计对我来说真是一次“痛并快乐着”的经历。从一开始画 ER 图，到后来写 SQL 各种报错，再到最后看到查询结果跑通，整个过程让我对数据库有了很多新的认识：

我们这次团队协作时在 github 上创建了仓库，使用 git 做文件和代码的版本控制，方便我们同步，另外我们使用上财的 Overleaf 写作创建报告。这次经历也丰富了我们的团队协作工具库。

我们在时间的时候也踩了很多意想不到的坑。印象最深的就是那个“序列自动跳号”的问题。明明操作是正确的，插入数据时主键 ID 却莫名其妙地跳过了几个数。查了半天资料也没解决。虽然只好将每个需要递增主键的表创建两遍才解决，但这个排错的过程让我对数据库的底层机制有了更深的印象。还有处理外键删除的时候，一开始没加 ON DELETE CASCADE，由于表与表之间的关系错综复杂，删起来很麻烦，加上级联删除后有了“一键扫荡”的爽快感。

另外，前期设计多花时间，后期写代码才能更省事。从 ER 图到转逻辑表到物理表，我们发现，前期的布局、深度思考与讨论是相当重要的。这才使得我们后续实践过程中，遇到的问题还是比较少的。

未来，如果可以继续这个项目的話，我们希望能够插入大量的真实数据测试，我们已经找到了一个 Github 上记录大量 AllRecipe 数据的仓库，可以处理之后加入到我们的数据库中作为测试数据。我们也希望能使用触发器和函数来实现我们提到的想法。最后希望能写一些“破坏性”的测试，看看数据库是不是足够健壮。

感谢一学期来老师的陪伴，数据库的学习给我们留下了很深刻的印象；同时也感谢老师的耐心审阅，期待老师的指正与指导。